

PRACTICAL WEEK-10

Task-1: Investigate inode structures using commands such as ls -li, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

```
root@Mani: ~$ practical$ ls -li original.txt
ls: cannot access 'original.txt': No such file or directory
root@Mani: ~$ practical$ echo "Hello Linux" > original.txt
root@Mani: ~$ practical$ ls -li original.txt
26209 original.txt
root@Mani: ~$ practical$ ln original.txt hardlink.txt
root@Mani: ~$ practical$ ls -li
total 296
26184 -rwxr-xr-x 1 the_boss the_boss 16248 Sep 19 03:50 address
26185 -rw-r--r-- 1 the_boss the_boss 703 Sep 19 03:49 address.c
26181 -rwxr-xr-x 1 the_boss the_boss 16512 Sep 19 03:35 client
26180 -rw-r--r-- 1 the_boss the_boss 819 Sep 19 03:34 client.c
26195 -rw-r--r-- 1 the_boss the_boss 79 Sep 25 11:11 copy.txt
26192 -rwxr-xr-x 1 the_boss the_boss 16288 Sep 19 04:00 cow
26194 -rw-r--r-- 1 the_boss the_boss 1015 Sep 19 04:00 cow.c
26209 -rw-r--r-- 2 the_boss the_boss 12 Sep 25 11:18 hardlink.txt
26163 -rw-r--r-- 1 the_boss the_boss 26 Sep 25 11:13 input.txt
26117 -rwxr-xr-x 1 the_boss the_boss 16224 Sep 25 11:03 lowlevel_copy
26187 -rw-r--r-- 1 the_boss the_boss 679 Sep 25 11:02 lowlevel_copy.c
26173 -rwxr-xr-x 1 the_boss the_boss 16304 Sep 19 03:30 ls_grep
26174 -rw-r--r-- 1 the_boss the_boss 804 Sep 19 03:30 ls_grep.c
26188 -rwxr-xr-x 1 the_boss the_boss 16080 Sep 19 03:52 maps
26189 -rw-r--r-- 1 the_boss the_boss 266 Sep 19 03:52 maps.c
26190 -rwxr-xr-x 1 the_boss the_boss 18104 Sep 19 03:58 memory
26191 -rw-r--r-- 1 the_boss the_boss 1080 Sep 19 03:57 memory.c
26209 -rw-r--r-- 2 the_boss the_boss 12 Sep 25 11:18 original.txt
26208 -rw-r--r-- 1 the_boss the_boss 26 Sep 25 11:14 output.txt
26110 -rwxr-xr-x 1 the_boss the_boss 16352 Sep 19 03:28 producer_consumer
26172 -rw-r--r-- 1 the_boss the_boss 997 Sep 19 03:27 producer_consumer.c
26205 -rwxr-xr-x 1 the_boss the_boss 16128 Sep 25 11:14 redirect
26207 -rw-r--r-- 1 the_boss the_boss 751 Sep 25 11:14 redirect.c
26196 -rwxr-xr-x 1 the_boss the_boss 16216 Sep 25 11:06 seek
26198 -rw-r--r-- 1 the_boss the_boss 374 Sep 25 11:06 seek.c
26178 -rwxr-xr-x 1 the_boss the_boss 16456 Sep 19 03:34 server
26179 -rw-r--r-- 1 the_boss the_boss 987 Sep 19 03:33 server.c
26182 -rwxr-xr-x 1 the_boss the_boss 16240 Sep 19 03:44 signal
26183 -rw-r--r-- 1 the_boss the_boss 597 Sep 19 03:43 signal.c
26203 -rwxr-xr-x 1 the_boss the_boss 16216 Sep 25 11:09 std_copy
26206 -rw-r--r-- 1 the_boss the_boss 607 Sep 25 11:08 std_copy.c
26204 -rw-r--r-- 1 the_boss the_boss 79 Sep 25 11:11 std_copy.txt
root@Mani: ~$ practical$ |
```

```
root@Mani: ~$ practical$ cat symlink.txt
Hello Linux
Added through hard link
root@Mani: ~$ practical$ rm original.txt
root@Mani: ~$ practical$ cat symlink.txt
cat: symlink.txt: No such file or directory
root@Mani: ~$ practical$ cat hardlink.txt
Hello Linux
Added through hard link
root@Mani: ~$ practical$ |
```

```

root@Mani: ~/practical$ find . -type f -printf "%i %p\n"
26209 ./hardlink.txt
26188 ./maps
26182 ./signal
26189 ./maps.c
26195 ./copy.txt
26163 ./input.txt
26117 ./lowlevel_copy
26173 ./ls_grep
26187 ./lowlevel_copy.c
26185 ./address.c
26207 ./redirect.c
26181 ./client
26194 ./cow.c
26191 ./memory.c
26179 ./server.c
26204 ./std_copy.txt
26184 ./address
26180 ./client.c
26208 ./output.txt
26172 ./producer_consumer.c
26206 ./std_copy.c
26183 ./signal.c
26192 ./cow
26198 ./seek.c
26178 ./server
26190 ./memory
26174 ./ls_grep.c
26110 ./producer_consumer
26205 ./redirect
26196 ./seek
26203 ./std_copy
root@Mani: ~/practical$

```

Report: The inode structure of Linux files was investigated using `ls -li`, `stat`, and `find`. A hard link was created and both filenames were found to share the same inode. A symbolic link was also created and was found to have a separate inode containing a reference to the original path.

The experiment demonstrated how hard and symbolic links affect inode allocation and file access.

Task-2 : Design a program using `mmap()` to perform file reading and writing. Compare memorymapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

Code:

```
GNU nano 8.7.1                                mmap_file.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    int fd;
    struct stat st;
    char *data;

    fd = open("input.txt", O_RDWR);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) == -1) {
        perror("fstat");
        close(fd);
        return 1;
    }

    data = mmap(NULL, st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    printf("Original content:\n");
    write(STDOUT_FILENO, data, st.st_size);
}
```

^G Help **^O** Write Out **^F** Where Is **^K** Cut **^T** Execute
^X Exit **^R** Read File **^I** Replace **^U** Paste **^J** Justify

```
root@Mani: ~/practical$ find . -inum 26209
./hardlink.txt
root@Mani: ~/practical$ echo "Hello from Linux memory mapped I/O!" > input.txt
root@Mani: ~/practical$ nano mmap_file.c
root@Mani: ~/practical$ gcc mmap_file.c -o mmap_file
root@Mani: ~/practical$ ./mmap_file
Original content:
Hello from Linux memory mapped I/O!

File modified using mmap().
root@Mani: ~/practical$ cat input.txt
HELLO from Linux memory mapped I/O!
root@Mani: ~/practical$ |
```

```
GNU nano 8.7.1                                traditional.c *
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd, out;
    char buffer[1024];
    ssize_t n;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    out = open("output.txt",
               O_WRONLY | O_CREAT | O_TRUNC,
               0644);

    if (out == -1) {
        perror("open");
        close(fd);
        return 1;
    }

    while ((n = read(fd, buffer, sizeof(buffer))) > 0)
        write(out, buffer, n);

    close(fd);
    close(out);

    printf("File copied using read()/write().\n");

    return 0;
}

|

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```

root@Mani: ~/practical$ nano traditional.c
root@Mani: ~/practical$ gcc traditional.c -o traditional
root@Mani: ~/practical$ ./traditional
File copied using read()/write().
root@Mani: ~/practical$ cat output.txt
HELLO from Linux memory mapped I/O!
root@Mani: ~/practical$ dd if=/dev/zero of=largefile bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.119422 s, 881 MB/s
root@Mani: ~/practical$ time ./mmap_file
Original content:
HELLO from Linux memory mapped I/O!

File modified using mmap().

real    0m0.008s
user    0m0.000s
sys     0m0.003s
root@Mani: ~/practical$ time ./traditional
File copied using read()/write().

real    0m0.003s
user    0m0.002s
sys     0m0.001s
root@Mani: ~/practical$ |

```

Report: The program was successfully implemented using `mmap()` for file reading and writing. The file was mapped into the process's virtual memory and modified directly through the mapped region.

A traditional `read()/write()` implementation was also developed for comparison. `mmap()` provides convenient direct memory-style access, while `read()/write()` is simpler to implement.

The efficiency of both methods depends on factors such as file size, caching, and access pattern.